

当文件被读取时，文件系统会先检查其内容是否已经保存在页缓存中。如果存在，则文件系统直接从页缓存中读取数据并将其返回应用程序。如果不存在，文件系统会在页缓存中创建新的内存页，从存储设备中读取相关的数据，并将其保存在创建的内存页中。之后，文件系统从内存页中读取相应的数据，并将其返回应用程序。

在进行文件修改时，文件系统同样首先检查页缓存。如果要修改的数据已经在页缓存中，文件系统可以直接修改页缓存中的数据，并将该页标记为脏页。若不存在，文件系统同样先创建页缓存并从存储设备中读取数据，此后在页缓存中进行修改并标记为脏页。标记为脏页的缓存由文件系统定期写回存储设备中。在操作系统内存不足或者应用程序调用 `fsync` 时，文件系统也会将脏页中的数据进行写回。

图 11.21 给出了页缓存中内存页的状态和转移关系。文件系统从磁盘中读出数据，创建新的缓存页。此时缓存页为干净页，表示其中的数据与存储中的数据相同。当干净页中的数据被修改时，干净页被标记为脏页，表示其中的数据与磁盘中的数据有所区别。当页中的数据被写回存储设备后，页面再次变为干净页，可以被回收以释放空间。

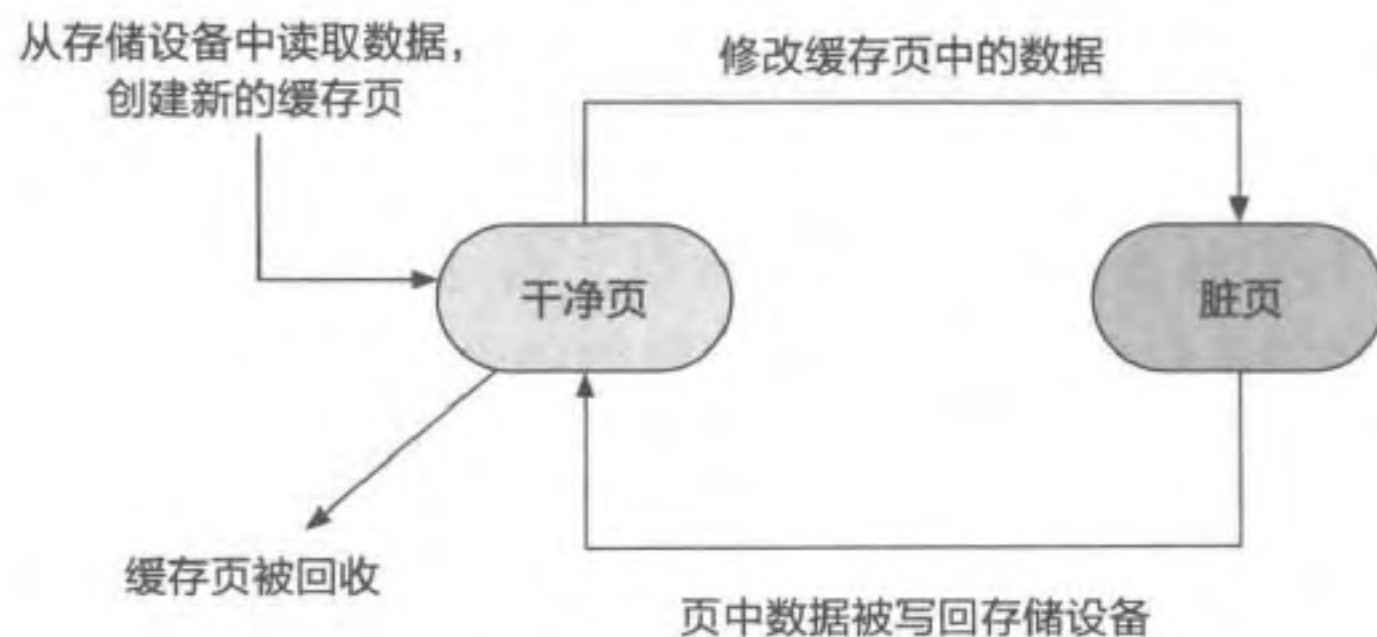


图 11.21 页缓存中内存页的状态和转移关系

小思考

页缓存一定能带来性能提升吗？

11.4.5 直接 I/O 和缓存 I/O

页缓存是持久化和性能之间的权衡。在大多数情况下，页缓存能够显著提升文件系统的性能，然而，在某些情况下，使用页缓存反而会起到负面作用。一种情况是，部分应用对数据持久化有较强的要求，不希望文件修改缓存在页缓存中。如果使用页缓存的机制，这些应用需要在每次修改文件后立即执行 `fsync` 操作进行同步，影响应用程序的性能。另一种情况是，一些应用程序（如数据库）自己实现缓存机制，以完全控制对数据的缓存和管理。由于应用程序更加了解自己对数据的需求，在这种情况下，操作系统提供的页缓存机制是冗余的，且一般会带来额外的性能开销。

因此，文件系统将是否使用页缓存的判断和选择权交给了应用程序。应用程序可以通过在打开文件时附带 `O_DIRECT` 标志，提示文件系统不要使用页缓存。这种文件访问方式就是直接 I/O (Direct I/O)，相对应的使用页缓存的文件请求被称为缓存 I/O (Cached I/O)。

11.4.6 内存映射

除了文件的 `read` 和 `write` 接口外，应用程序还可以通过内存映射机制以访问内存的形式访问文件内容。Linux 在其页缓存的基础上实现文件的内存映射机制。

在建立内存映射前，应用程序先打开目标文件，获得其文件描述符，随后调用 `mmap` 接口建立文件内存映射。调用时提供映射的目标虚拟地址^①、长度、属性、标志位、文件描述符和起始位置在文件中的偏移量。

在处理内存映射请求时，VFS 会分配对应的 VMA 结构，并通过在 VMA 结构中记录目标文件的 `inode` 和映射时的属性，将 VMA 对应的虚拟地址空间与文件 `inode` 进行关联，最后返回起始虚拟地址给应用程序。由于此时并未更新页表，当应用程序首次访问映射后的虚拟地址时，会触发缺页异常。Linux 在处理缺页异常时，会根据 VMA 中记录的 `inode` 信息，调用对应的文件系统进行处理。文件系统将从文件的页缓存中找到对应的内存页并返回给 VFS。VFS 最终将页缓存中内存页的物理地址填入页表，返回应用程序继续执行。由于页表中的映射已经建立，应用程序对同一个虚拟页的后续访问不会再触发缺页异常。

需要注意的是，在进行内存映射之后，应用程序在进行访问时，访问的是页缓存对应的内存页。其修改的数据保存在页缓存中，可能并未写回存储设备^②。因此，为了保证修改的数据写回存储设备上，应用程序需要调用 `msync` 接口请求 VFS 对指定内存映射区域进行同步写回操作。

当所有操作完成之后，应用程序可以调用 `munmap` 移除指定虚拟内存地址区域上的内存映射。

11.5 用户态文件系统

本节主要知识点

- ❑ 用户态文件系统有哪些优点？
- ❑ FUSE 是如何允许应用程序访问用户态的文件系统服务的？
- ❑ ChCore 是如何管理和协调多个文件系统的？
- ❑ ChCore 是如何实现一个简单的内存文件系统的？

① 需配合 `MAP_FIXED` 标志位使用，否则映射的目标虚拟地址由操作系统决定。

② 之所以说“可能”，是因为操作系统可能由于其他原因（如内存不足或其他应用调用了 `fsync`）已将部分页缓存中的数据修改写回存储设备。

11.5.1 为什么需要用户态文件系统

对于宏内核来说，其文件系统一般实现在内核态，作为内核的一部分或者内核模块，为用户态应用程序提供服务。然而在内核态实现文件系统有诸多不便之处。第一，内核态的文件系统与内核其他部分是强相关的。它们具有相同的运行环境和地址空间，内核文件系统上的漏洞或缺陷会影响内核的其他部分，并且影响整个操作系统的功能、性能和安全。第二，为了保证内核代码的安全和可靠性，在内核中实现的文件系统不宜加入大量的第三方代码。同时，由于构建方法等限制，将各种不同的代码混合编译到内核也是不现实的。因此，内核态文件系统无法使用用户态的大量第三方代码，能复用的代码有限。第三，内核中实现的文件系统缺少像用户态中那样方便易用的调试工具。因而在内核态进行文件系统的调试难度较大，开发效率较低。

鉴于这些问题，在用户态实现文件系统成为一个很好的选择。由于用户态应用程序受到内核的监控和限制，用户态文件中出现的缺陷和漏洞不会影响内核以及其他用户态应用程序。因而在开发和调试用户态文件系统时，如果发生错误，开发者只需要重启其所在的应用程序即可，无须重启整个系统。此外，用户态文件系统能以源代码、链接库等方式方便地使用第三方代码，能够用 `gdb` 等工具进行调试，还能与浏览器、邮件服务等其他软件进行协作，实现诸如浏览网页、收发邮件等额外功能。

用户态文件系统的思想可以被看作微内核思想在文件系统上的一种延伸。原本在内核中实现的文件系统组件（包括 VFS 和具体的文件系统实现），可以被划分到用户态作为系统服务运行。在后文中，我们将分别介绍 Linux 中的 FUSE 模块和框架^[11-12]以及微内核系统中的文件系统设计。前者在 Linux 宏内核的系统调用基础上引入一系列机制，允许在用户态的文件系统服务中完成其他应用对文件系统的访问；后者则进一步基于微内核 IPC 将 VFS 相关逻辑也移到用户态进行。

11.5.2 FUSE

FUSE (Filesystem in USErspace) 是一个允许在用户态实现文件系统的框架。图 11.22 中展示了 Linux 宏内核中的 FUSE 架构。可以看出，应用程序依然通过系统调用与内核中的 VFS 模块进行交互。若 VFS 发现请求的路径由某个文件系统（如 Ext4 文件系统）管理，会根据请求的内容，将请求交由相应的文件系统进行处理。文件系统则通过块抽象层和 I/O 调度等，最终在存储设备中访问数据。若 VFS 发现应用程序请求的路径由 FUSE 模块管理，会将请求交给 FUSE 模块（图中的彩色箭头），FUSE 模块会再根据其记录的信息将请求加入对应的共享队列中。在用户态运行的 FUSE 文件系统服务器是一个用户态应用程序。该应用程序在开始运行时，会向内核中的 FUSE 模块进行注册，此后不断地从共享队列中获取文件请求并进行处理。处理的结果通过队列返回 FUSE 模块，FUSE 模块拿到处理结果后，再将其返回发出文件请求的应用程序。

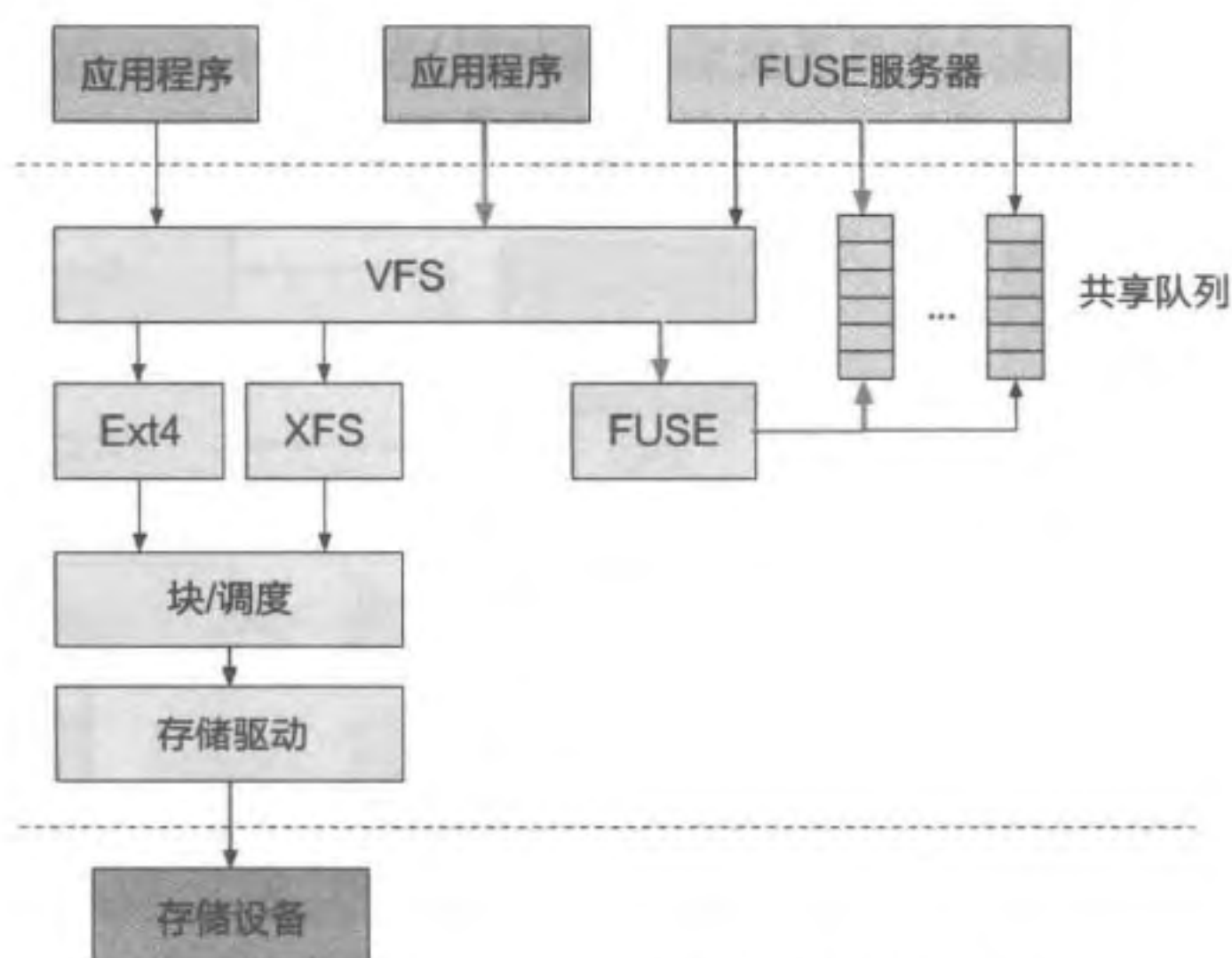


图 11.22 Linux 宏内核中的 FUSE 架构

用户态的 FUSE 文件系统服务器可以通过各种方式处理应用程序的文件请求，如在内存中维护一个内存文件系统来保存数据，通过系统调用使用内核中其他文件系统和存储设备上的数据，或者访问网络中其他设备上的存储资源。在用户态实现文件系统，还便于将其他不同资源整合成文件抽象。例如，可以将电子邮件变成 FUSE 文件系统服务器所管理的资源，使应用程序或者用户可以通过访问文件，实现对电子邮件的接收、查看、编辑和发送等操作。

虽然位于用户态的 FUSE 文件系统极大地提高了灵活性，但也存在诸多问题，性能是其中最为突出的一个。从图 11.22 中可以看到，访问一个普通的内核文件系统中的文件，应用程序的请求只需要进入内核态，由文件系统访问存储设备即可直接完成。然而对于一个用户态文件系统的访问来说，其访问请求会在用户态和内核态之间反复转发。这种频繁的进程切换、请求转发和数据拷贝，会极大地影响用户态文件系统的性能。因此基于 FUSE 的用户态文件系统一般会用在一些性能并非十分重要的场景，或者用来研究和试验新的文件系统设计。

11.5.3 ChCore 的文件系统架构

在微内核架构下，文件系统服务一般会被放在用户态。除了具体的文件系统实现之外，微内核的 VFS 部分也会在用户态中实现。在 ChCore 中，文件系统管理器作为用户态的 VFS 来提供统一的文件系统服务并管理多个文件系统实例。

图 11.23 展示了 ChCore 中的应用程序、文件系统管理器和文件系统实例之间的关系。应用程序将文件系统的请求通过 IPC 发送给文件系统管理器。文件系统管理器则根据具体的文件系统请求进行相应处理，并按需将请求发送给对应的文件系统实例。若所有请求都经过文件系统管理器进行转发，则需要多次 IPC 请求，性能开销较大。因此，

对于数据请求，应用程序通过文件系统管理器与对应的文件系统直接建立 IPC 连接（图中的黑色箭头），减少请求过程中 IPC 的数量，提升系统的性能。

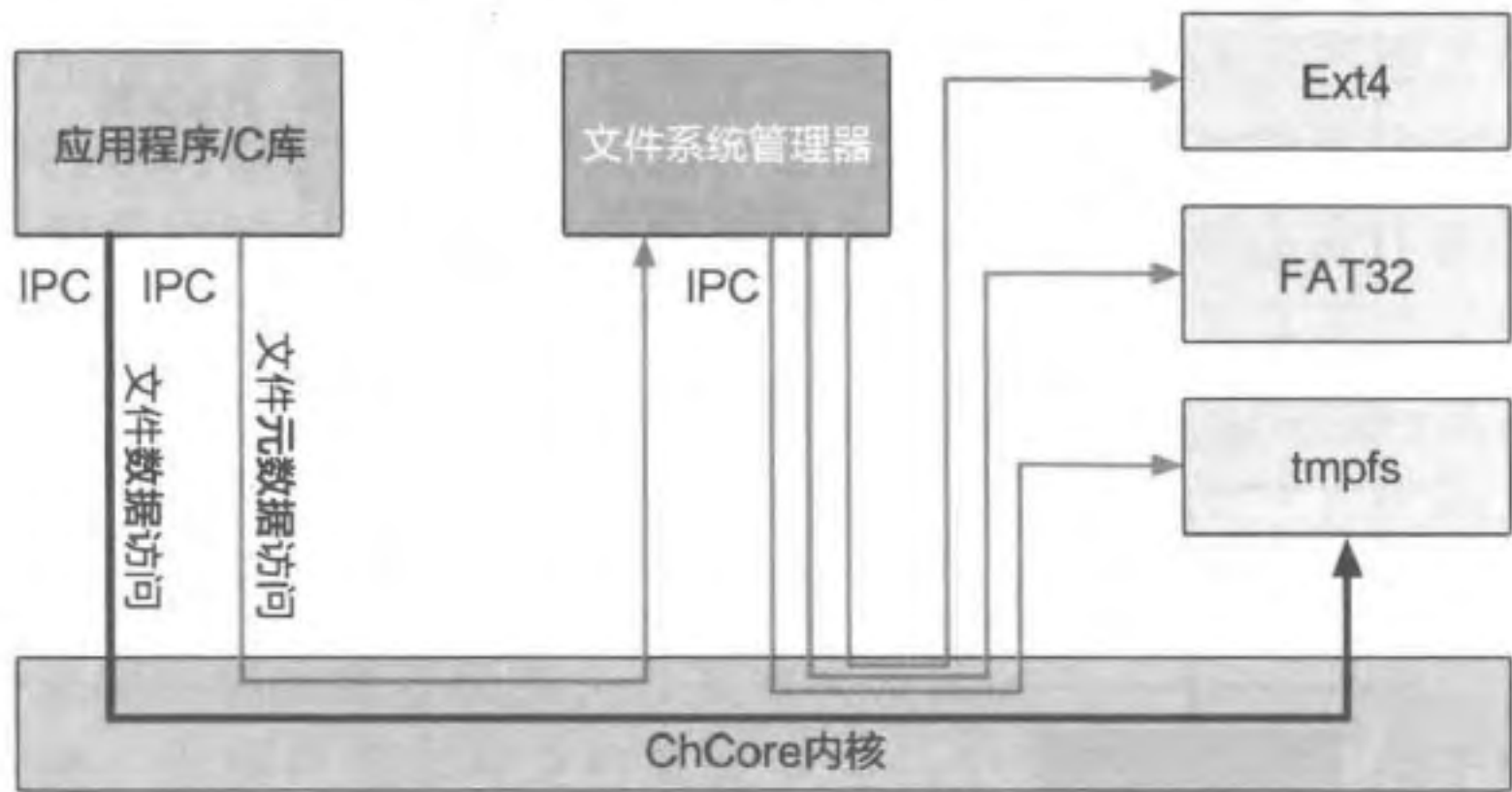


图 11.23 ChCore 中的文件系统

这种设计是操作系统设计中一种比较常用的方法：数据面和控制面分离。这种方法往往基于两个观察：在应用运行过程中，数据面的请求数量要远大于控制面的请求数量，因此提升数据面请求的处理速度是十分有效的；控制面在进行处理时往往需要非常复杂的处理逻辑，需要比较久的时间。数据面仅仅是对数据进行简单的处理和传输，可以快速进行。将数据面和控制面分开，有助于数据请求更快地得到处理，从而提升系统性能。

多文件系统支持

文件系统管理器还需要管理多种文件系统实例。通过文件系统管理器在中间作为协调，应用程序可以用统一的接口来访问不同的文件系统。作为管理者，文件系统管理器维护了应用程序能够看到的整个视图，并通过解析应用程序发出的请求，将请求交由具体的文件系统服务进行处理。

文件描述符和当前工作目录的维护

由于每个应用程序在访问文件系统元数据时均会通过文件系统管理器，ChCore 在文件系统管理器中维护每个进程的文件系统描述符和当前工作目录。

与前文中介绍的设计相似，ChCore 的文件系统管理器为每个文件描述符保存了一个文件描述结构，其中保存了文件打开模式和文件当前的读写位置信息。不同的是，由于文件系统管理器并不使用 inode 作为文件的管理结构，其文件描述结构中并没有保存 inode，而是保存了文件所在的文件系统服务器和文件在服务器中的标识符。通过这两个信息，文件系统管理器在处理文件请求时能与正确的文件系统服务器通信，共同完成应用程序的文件请求。除了文件系统描述符外，每个进程的当前工作目录同样在文件系统管理器中进行维护。当使用相对路径进行路径解析时，文件系统管理器直接从其维护的当前工作目录开始进行文件查找操作。

内存文件系统

现有的 ChCore 设计中包含一个简单的内存文件系统，用于保存系统中的临时文件。

内存文件系统将数据保存在内存中，不考虑数据的持久化问题，因此其设计和实现比传统的持久化文件系统更加简单。图 11.24 展示了 ChCore 内存文件系统的主要结构。

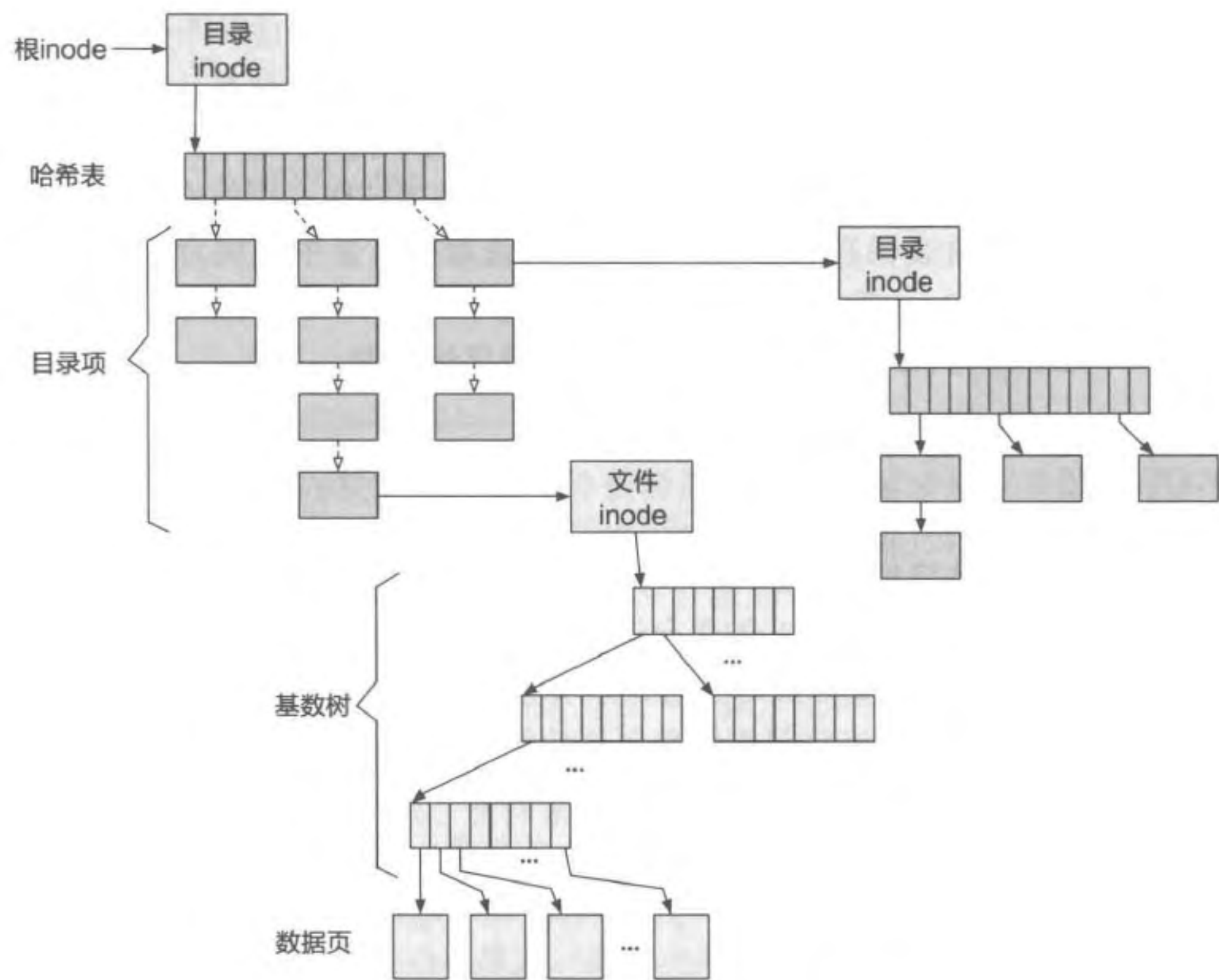


图 11.24 ChCore 中的内存文件系统

ChCore 内存文件系统的主要组成部分及功能如下。

- 存储布局与超级块。ChCore 中的内存文件系统不需要持久化，因此并没有存储布局的设计。但是该内存文件系统有超级块结构，超级块保存在内存中，记录当前文件系统的整体状况和全局信息，包括整个文件系统中保存的文件数量，整个文件系统占用的内存大小等。
- inode 结构。ChCore 的内存文件系统同样基于 inode。其 inode 中保存对应的文件类型、链接数、文件大小等通用元数据。对于常规文件和目录文件，还分别保存基数树的根节点和哈希表。
- 文件结构。ChCore 的内存文件系统中，每个文件使用一棵基数树管理文件中所有的数据页，其中数据页的序号作为键，内存页的虚拟地址作为值。ChCore 并没有采取图 11.5 中的间接块方式组织数据页，这主要是为了复用现有的基数树的实现。由于内存文件系统在用户态实现，因此可以使用许多用户态类库。举例来

说，若使用 C++ 或者 Rust 等语言编写内存文件系统，可以利用 `std::map` 或者 `std::collections::HashMap` 实现文件数据的组织。当然，一种更简单直接的方法是使用数组或者 `std::vector` 等结构保存文件数据。

- 目录结构。ChCore 内存文件系统 中的每个目录文件均对应一个哈希表。哈希表中保存的是此目录下的所有子文件目录项。哈希表以文件名（以及文件名的哈希值）作为键，值保存了目录项。目录项中包括完整的文件名，以及该文件名对应的 `inode` 指针。相对于文件数据来说，目录项结构占用的内存更小且长度不固定，使用哈希表保存目录项，可以更快速地在目录中找到对应的目录项。若使用 C++ 或者 Rust 等语言编写内存文件系统，同样可以利用 `std::map` 或者 `std::collections::HashMap` 实现目录项的组织。

11.6 思考题

1. 在介绍文件的符号链接和硬链接时，我们提到硬链接的目标文件不能为目录。为什么硬链接的目标文件不能是目录？如果允许硬链接到一个目录文件，会产生什么问题？如果一定要设计一个支持目录硬链接的文件系统，应如何设计和实现？
2. 在云环境下，有许多存储方法只提供单层文件管理，即不支持目录。这种设计有哪些优点？在这种情况下，如果用户或者应用程序需要“目录”功能来更好地管理自己的文件，那么该如何操作才能达到模拟“目录”的效果？
3. 文件描述符的本质也是一种 `Capability`。在微内核的情况下，能否直接使用 `Capability` 替代文件描述符？如果可以，会有哪些挑战？如果不行，为什么？
4. 当存在“`/home/chcore`”目录，其中却不存在“`filesystem.tex`”文件时，运行代码片段 11.7 中的程序会打印什么信息？如果多次运行呢？为什么会有这种现象？

代码片段 11.7 使用文件接口读写文件的一段程序

```
#include <fcntl.h>
#include <stdio.h>
#include <unistd.h>

#define DATA_SIZE 20

int main()
{
    int fd;
    char data[DATA_SIZE + 1];

    // 打开文件
    fd = open("/home/chcore/filesystem.tex",
              O_RDWR | O_CREAT, S_IRUSR | S_IWUSR);
```